UCLL
HOGESCHOOL

Full-Stack software development

Lab 1: Typescript & testing

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Contents

1 Introduction

In this lab, you will learn the core principles of TypeScript and unit testing with Jest by building an interactive Shopping Cart.

The concepts covered include:

- Classes, interfaces and properties
- Optional (?) and null types
- Custom types and enums
- Passing functions as parameters (callbacks)
- Testing happy and unhappy scenarios with Jest
- Compiling a TypeScript application
- Running a TypeScript application in console and browser

In addition to the presentation slides, you can use the links below to successfully complete this lab:

- https://www.typescriptlang.org
- https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html
- https://jestjs.io/
- https://jestjs.io/docs/getting-started
- https://jestjs.io/docs/using-matchers

2 Getting started

To begin this lab, please accept the following GitHub assignment:
https://classroom.github.com/a/F1bRLzok

Once you have done this, you will be provided with a repository containing startcode. Clone it to a local directory and open it in VSCode.

Carefully read the instructions in README.md first! These instructions explain how to install Node.js, Node Package Manager (npm) and how to install all necessary dependencies to run this lab.

3 Product

What you will learn

3.1

- ✓ How to define a class with properties.
- ✓ How to add validation logic to enforce business rules.
- ✓ How to handle optional fields (?) and test for undefined.
- ✓ How to test for expected errors using Jest's .toThrow() matcher.

Tasks

3.2 You are provided with a very minimal Product class. We will expand this into a domain object where state is modified through properties.

3.2.1 PROPERTIES

Define properties for id (number), name (string), price (number) and an optional description (string). Declare properties as private and prefix them with an underscore (_) to ensure they are properly encapsulated.

Learn how to use properties in TypeScript on https://www.typescripttutorial.net/typescript-tutorial/typescript-getters-setters.

Remember to use the ? symbol in the type definition to mark a field as optional

3.2.2 SET AND GET PROPERTIES

Modifications to properties should always be made using *set* methods, which can contain validation logic. To read properties, always use *get* methods.

Implement the *set name* function:

- Accepts a name of type String parameter.
- It should throw an Error with the message "Name cannot be empty." if the name is an empty string. Otherwise, it updates the name property.

Implement the *set price* function:

- Accepts a price of type number parameter.
- It should throw an Error with the message "Price cannot be negative." if the price is less than 0. Otherwise, it updates the price property.

Implement *set description* function:

- It should accept a parameter description of type string
- It should set the optional description.

Finally, for every property, implement a *get* function.

3.2.3 CONSTRUCTOR

The constructor should only accept the essential parameters required to create a valid initial state: id, name, and price. It should always call the set method to initialize the properties, ensuring validation is run upon creation.

3.2.4 TESTING:

In *product.test.ts*, write the following tests:

- A test that validates a product is created correctly via the constructor.
- Tests that validate the setName and setPrice methods successfully update the properties with valid data.
- A test that validates that for a product created without a description, getDescription() returns undefined.
- A test that validates that calling setPrice() with a negative value throws an error.
- A test that validates that calling setName() with an empty string throws an error.

To test for an error, wrap the code that should throw it in a lambda function: expect(() => object.method(…)).toThrow('…').

Now that you have written your first tests, it's time to run them. Jest is the test runner for this project and is already configured. To execute all test files (ending in .test.ts) in the project, simply run the fllowing command in your terminal:

```
npm run test
```

Jest will automatically find and run the tests, providing you with a summary of which tests passed and which failed. Use this command whenever you want to validate that your code works as expected.

4 ShoppingCart

What you will learn

4.1

- ✓ How to manage a collection of objects within a class.
- ✓ How to use union types (| null) in a method's return value to handle cases where an object is not found.
- ✓ How to write tests for methods that can return null.

Tasks

4.2

4.2.1 IMPLEMENTATION

Define the ShoppingCart class in *shoppingCart.ts*:

- The class has a private property *items* of type Product[], initialized as an empty array.
- Implement the following public methods:
  - addProduct(product: Product): void
  - removeProduct(productId: number): Product | null

    This method finds a product by id. If found, it is removed and returned. If not found, it returns null.
  - getTotalPrice(): number
  - get items(): Product[]

4.2.2 TESTING

In *shoppingCart.test.ts*, test the behavior of removeProduct:

- Write a test that validates the correct product is returned after a successful removal.
- Write a test that validates *null* is returned when trying to remove a non-existent product (use the *.toBeNull()* matcher).
- You only need to implement the tests related to removal for now.

5 Refactor with type and enum

What you will learn

5.1

- ✓ How to create and use custom type aliases for more descriptive and complex data structures.
- ✓ How to use enum to represent a fixed set of states, making your code more readable and less error-prone.
- ✓ How to refactor existing code to meet new requirements.

Tasks

5.2

5.2.1 TYPE DEFINITIONS

In the folder *types*, open the file *index.ts*. Define 2 new types:

- A type CartItem as { product: Product; quantity: number; }
- An enum CartStatus with the values Active, Checkout, Paid.

5.2.2 REFACTOR SHOPPINGCART

- Change the *items* property from Product[] to CartItem[].
- Add a *status* property of type CartStatus, defaulting to Active.
- Rewrite addProduct: if a product is already in the cart, increment its quantity. Otherwise, add a new CartItem.

Before adding a new item, use a method like find() to check if an item with the same product ID already exists in the cart.

- Update getTotalPrice to account for quantity.

6 Working with callbacks

What you will learn

6.1

- ✓ How to define the type signature for a function that is passed as a parameter (a callback).
- ✓ How to write a higher-order function that uses a callback to perform a generic action.
- ✓ How to use lambda expressions to provide an inline implementation for a callback.

Tasks

6.2

6.2.1 IMPLEMENTATION

Add a search function to the ShoppingCart class that uses a callback function.

- Add a new public method *findCartItem* to the ShoppingCart class.
- This method accepts one parameter:
  - predicate: a callback function with the signature (item: CartItem) => boolean.
- The findCartItem method should return the found CartItem, or null.

The built-in this.items.find(predicate) returns undefined when an item is not found. You need to convert this result to null. The nullish coalescing operator (??) is perfect for this: return this.items.find(predicate) ?? null;

Good to Know: Is the predicate a callback function?
Yes, it is. A callback function is any function that is passed as an argument to another function, which will then “call it back” at an appropriate time. In this case, the findCartItem method calls your predicate function back for each item in the array to ask, “Is this the item we're looking for?” While the term is often associated with asynchronous operations (like fetching data), the

pattern is identical for synchronous code like this. You will see this concept used extensively in React for handling events (e.g. onClick, onChange).

6.2.2 TESTING

Write a tests for *findCartItem*:

- Test the happy path: call the method with a predicate that finds an existing item and assert that the correct item is returned.
- Test the unhappy path: call the method with a predicate that does not find any item and assert that the result is null (using .toBeNull()).
- Write a test for adding a product that already exists in the shopping cart, where its quantity is increased by one instead of the item being added twice.

7 Running the application

What you will learn

7.1

- ✓ The difference between running TypeScript in a server-side (Node.js) and client-side (browser) environment.
- ✓ How to use a build tool to compile TypeScript to JavaScript that the browser can understand.

Tasks

7.2 Now that all the logic for our domain objects is complete and tested, it's time to see it in action. We will create a main.ts file to act as the entry point for our application and run it in two different JavaScript environments.

7.2.1 CREATE THE MAIN SCRIPT

Create a file named main.ts in the root of your source folder. This file will orchestrate your application.

Import everything you need (Product, ShoppingCart and CartStatus) and write a *displayCart* function:

```
const displayCart = (cart: ShoppingCart): void => {
  console.log('--- Shopping Cart Status ---');

  cart.items.forEach((item) => {
    console.log(
      `- ${item.product.name} (${item.quantity}): ${item.product.price}`
    );
  });
  console.log('Total Price:', cart.getTotalPrice());
  console.log('----------------------------');
};
```

Build the main application logic:

```
// Create a new shopping cart
const cart = new ShoppingCart();

// Create some products
const laptop = new Product(1, 'Laptop', 999);
const mouse = new Product(2, 'Mouse', 25);
laptop.description = 'A powerful laptop for all your needs.';

console.log('Initial state:');
displayCart(cart);

console.log('\nAdding laptop...');
cart.addProduct(laptop);
displayCart(cart);

console.log('\nAdding another laptop...');
cart.addProduct(laptop);
displayCart(cart);

console.log('\nAdding mouse...');
cart.addProduct(mouse);
displayCart(cart);

console.log('\nRemoving laptop...');
cart.removeProduct(1);
displayCart(cart);
```

7.2.2 RUN IN THE CONSOLE (NODE.JS ENVIRONMENT)

First, we'll run the script directly in a Node.js environment using ts-node. This tool compiles and runs your TypeScript code in memory without creating any visible .js files.

1. Open your terminal in VSCode or Git Bash.
2. Run the following command:

```
npx ts-node src/main.ts
```

3. Observe the output: You should see the step-by-step logs from your main.ts script printed directly in your terminal. This confirms your code works as expected on the "server-side."

7.2.3 RUN IN THE BROWSER ("CLIENT-SIDE")

Right now, you are viewing your application's output in the developer console. The data you see - the list of cart items and the total price - is known as the application's *state.*
In a real web application, you wouldn't just log this state to the console. Instead, you would use it to dynamically generate HTML to show the shopping cart to the user.
This is where frameworks like *React* come in. In the upcoming weeks, you will learn how to use React to take this state data and "render" it as a user interface. React will automatically update the HTML on the page whenever the state changes (e.g., when a new product is added).

Browsers cannot execute TypeScript directly. Therefore, we must first compile (transpile) our TypeScript (.ts) files into standard JavaScript (.js) files that any browser can understand.
Compile Your Code by running the build script from your package.json.

```
npm run build
```

What does npm run build do?

This command tells npm (Node Package Manager) to look inside your package.json file for a script named "build". This script typically looks something like this:

```
"scripts": {
  "build": "tsc"
}
```

- npm executes the command defined in the script, in this case *tsc*.
- tsc is the TypeScript Compiler. It reads its configuration from your *tsconfig.json* file.

- This configuration file tells the compiler where your TypeScript source files are and where to put the compiled JavaScript output (e.g. in a /dist directory).
- The result is a set of .js files in the output directory that are ready to be run by a browser.

Next, create an *index.html* file in the project's root directory that hosts our application:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Lab 1: Shopping Cart</title>
    <script type="module" src="./dist/src/main.js" defer></script>
  </head>
  <body>
    <h1>Shopping Cart Application</h1>
    <p>Open the developer console (F12) to see the output.</p>
  </body>
</html>
```

To load your application, the index.html file uses a single <script> tag that points to your compiled code at src="./dist/src/main.js". This tag includes two essential attributes: type="module" is required for the browser to understand the import and export syntax used to structure our JavaScript code. The defer attribute ensures the script only executes after the entire HTML document has been parsed.

Finally, open the index.html file in your web browser with the Live Server extension, then press F12 to open the Developer Tools and view the Console tab. You should see the exact same logs as you saw in your terminal.